EVENT PARKING RESERVATION SYSTEM

How to Clone, Configure, Build and Run the GitHub Project on Another PC

ASP.NET Core 8 Web API + SQL Server LocalDB + Static HTML/CSS/JavaScript Frontend

Repository	CodeEver-CEO/Event-Parking-Reservation-System-Team
Recommended branch	develop
Backend framework	.NET 8 / ASP.NET Core Web API
Database	SQL Server LocalDB (default) / SQL Server
API documentation	Swagger / OpenAPI
Guide basis	Uploaded develop-branch source snapshot (11 Aug 2026)

Prepared as a practical handover / setup document for development and testing.

Quick Start - One Page

Success: For a new Windows PC, the shortest reliable path is: install prerequisites -> clone develop -> configure JWT secret -> restore/build -> run backend -> confirm Swagger -> configure/serve frontend.

Step	Action	What to do
1	**Install**	Git, .NET SDK 8.0, and SQL Server LocalDB (usually through Visual Studio / SQL Server Express).
2	**Clone**	Clone the GitHub repository and switch to the develop branch.
3	**JWT secret**	Create a Base64 32-byte key and save it with .NET user-secrets.
4	**Database**	Use the default LocalDB connection or replace it with a valid SQL Server connection string.
5	**Build**	Run dotnet restore and dotnet build on backend/EventParkingReservationSystem.sln.
6	**Run API**	Run the API with the https launch profile.
7	**Verify**	Open Swagger at https://localhost:7239/swagger or http://localhost:5197/swagger.
8	**Frontend**	Change frontend/js/config.js from port 7001 to the API port, then serve the frontend folder.

```
git clone -b develop https://github.com/CodeEver-CEO/Event-Parking-Reservation-System-Team.git
cd Event-Parking-Reservation-System-Team
dotnet restore .\backend\EventParkingReservationSystem.sln
dotnet build .\backend\EventParkingReservationSystem.sln
```

1. What Is in This Project?

The repository contains an ASP.NET Core Web API backend, a static frontend, and a database folder. The uploaded source snapshot includes controllers and services for authentication, admin authentication, customers, events, venues, categories, bookings, seats, parking, payments and notifications.

```
Event-Parking-Reservation-System-Team/
 backend/
  EventParkingReservationSystem.sln
  EventParkingReservationSystem.API/
 frontend/
  index.html
  js/
  css/
  pages/
 database/
  EventParkingReservationSystem.sql
```

2. Prerequisites on the New PC

- Windows 10/11 is the easiest environment because the default connection uses SQL Server LocalDB.
- Git for Windows - required to clone and update the GitHub repository.
- .NET SDK 8.0 - the project targets net8.0.
- SQL Server LocalDB, normally installed with Visual Studio or SQL Server Express.
- Recommended IDE: Visual Studio with ASP.NET and web development workload. Visual Studio Code can also be used.
- Optional: dotnet-ef CLI if you want to inspect or manually apply EF Core migrations.

```
git --version
dotnet --version
sqllocaldb info
```

Important: If dotnet --version does not show an 8.x SDK, install .NET SDK 8 before continuing.

3. Clone the GitHub Repository

Open PowerShell in the folder where you want the project. Clone the develop branch directly:

```
git clone -b develop https://github.com/CodeEver-CEO/Event-Parking-Reservation-System-Team.git
cd Event-Parking-Reservation-System-Team
```

Confirm that you are on the correct branch and that the working tree is clean:

```
git branch --show-current
git status
```

Note: Expected branch: develop. If the repository is private, GitHub may ask you to authenticate.

4. Configure the JWT Signing Key - Required

The repository intentionally does not keep the JWT signing secret in appsettings.json. A fresh clone therefore needs a local user secret before the API can start correctly.

From the project root, enter the API project folder:

```
cd .\backend\EventParkingReservationSystem.API
```

Generate a random 32-byte Base64 key in PowerShell:

```
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Max 256 }))
```

Copy the generated value and store it for this project:

```
dotnet user-secrets set "Jwt:Key" "PASTE_THE_GENERATED_BASE64_KEY_HERE"
```

Verify the secret is registered:

```
dotnet user-secrets list
```

Important: User secrets are stored per user/per PC and do not travel with Git. Every new development PC must configure Jwt:Key again.

5. Database Setup

The default connection string in appsettings.json uses SQL Server LocalDB and creates/uses the database EventParkingReservationSystemDb:

```
Server=(localdb)
\MSSQLLocalDB;Database=EventParkingReservationSystemDb;Trusted_Connection=True;TrustServerCertificate=True;
```

5.1 Check or start LocalDB

```
sqllocaldb info
sqllocaldb start MSSQLLocalDB
```

5.2 EF Core migrations

The current Program.cs attempts to start LocalDB, runs pending migrations automatically in Development, and seeds the default administrator. Therefore, for a normal fresh development PC, running the API can initialize the database automatically.

If you prefer to apply migrations manually first:

```
dotnet tool install --global dotnet-ef --version 8.0.0
dotnet ef database update
```

Warning: The uploaded snapshot contains an InitialCreate migration and ApplicationDbContextModelSnapshot.cs. Do not delete or regenerate migrations casually when joining the project; coordinate migration changes with the team.

6. Development Admin Account

In Development, DatabaseSeeder reads the AdminSeed section from appsettings.Development.json and creates the administrator only if it does not already exist.

Name	System Administrator
Email	admin@eventparking.local
Password	Admin@12345

Warning: These are development seed credentials only. Do not use these credentials in production, and do not publish production secrets in source control.

7. Restore and Build the Backend

Return to the repository root if needed, then restore packages and build the solution:

```
cd ..\..
dotnet restore .\backend\EventParkingReservationSystem.sln
dotnet build .\backend\EventParkingReservationSystem.sln
```

Success: A successful result should end with "Build succeeded". Fix compile errors before trying to run the API.

8. Trust the Local HTTPS Development Certificate

If the browser warns about the ASP.NET Core development certificate, run:

```
dotnet dev-certs https --trust
```

Accept the Windows confirmation dialog, then restart the API/browser if necessary.

9. Run the Backend API

From the repository root, run the API using the HTTPS launch profile:

```
dotnet run --project .\backend\EventParkingReservationSystem.API\EventParkingReservationSystem.API.csproj --launch-profile https
```

The checked-in launchSettings.json defines these development URLs:

- HTTPS: https://localhost:7239
- HTTP: http://localhost:5197
- Swagger: https://localhost:7239/swagger or http://localhost:5197/swagger

A successful startup normally shows messages similar to:

```
Now listening on: https://localhost:7239
Now listening on: http://localhost:5197
Application started. Press Ctrl+C to shut down.
```

Note: To stop the API cleanly, return to the terminal and press Ctrl+C before rebuilding or switching branches.

10. Verify the Backend in Swagger

Open Swagger and confirm that the main controller groups are visible. In the uploaded snapshot, the backend includes routes for the following areas:

- AdminAuth
- Auth
- Bookings
- Categories
- Customers
- Events
- Notifications
- ParkingReservations
- ParkingSlots
- Payments
- Seats
- Venues

For protected APIs, login first, copy the JWT access token, select Authorize in Swagger, and enter the token. Customer-only and Admin-only routes should return 401/403 when the wrong authentication context is used.

11. Configure and Run the Frontend

The frontend is static HTML/CSS/JavaScript under frontend/. One important configuration change is required in the uploaded snapshot: frontend/js/config.js currently points to port 7001, while the backend launch settings use ports 7239 and 5197.

Open:

```
frontend\js\config.js
```

Change:

```
API_BASE_URL: "https://localhost:7001/api",
```

to one of the actual backend URLs, preferably:

```
API_BASE_URL: "https://localhost:7239/api",
```

If HTTPS certificate setup is a problem during local development, use:

```
API_BASE_URL: "http://localhost:5197/api",
```

Important: This port alignment is essential. If the frontend still points to https://localhost:7001/api, requests will fail even when the backend is running correctly.

11.1 Serve the static frontend

You can use Visual Studio Code Live Server, another static web server, or Python if it is installed:

```
python -m http.server 5500 -d frontend
```

Then open:

```
http://localhost:5500
```

12. How to Get the Latest GitHub Code Later

Before pulling, make sure you do not have uncommitted work. For a clean develop branch:

```
git status
git switch develop
git fetch origin
git pull --ff-only origin develop
```

Note: git fetch downloads remote history without merging it into your working files. git pull --ff-only updates only when Git can fast-forward cleanly, avoiding an unnecessary merge commit.

13. Common Problems and Fixes

Problem	Recommended action
"dotnet" is not recognized	Install .NET SDK 8.0 and reopen PowerShell. Confirm with dotnet --version.
JWT key is required / API stops at startup	Run dotnet user-secrets set "Jwt:Key" "<Base64 key>" inside the API project folder.
Jwt:Key must be valid Base64	Generate the key with the PowerShell command in Section 4; do not type an arbitrary plain-text password.
LocalDB / SQL connection error	Confirm SQL Server LocalDB is installed, then run sqllocaldb start MSSQLLocalDB. Check the DefaultConnection string.
dotnet ef command not found	Install the EF CLI: dotnet tool install --global dotnet-ef --version 8.0.0.
Browser HTTPS certificate warning	Run dotnet dev-certs https --trust or temporarily use the HTTP launch URL for local testing.
Frontend cannot call API	Confirm the API is running and change frontend/js/config.js from port 7001 to 7239 (HTTPS) or 5197 (HTTP).
Build says the .exe is being used by another process	Stop the running API with Ctrl+C, then build again.

Build succeeds but Windows blocks the generated DLL (0x800711C7)	This can be an enterprise Windows Application Control policy. Do not disable organizational security controls; use an approved developer environment or contact the PC administrator/IT team.
Database schema errors on a fresh PC	Check that the current migrations are present and run dotnet ef database update. Avoid manually editing migration history.

14. Final Verification Checklist

☐ Git branch shows develop (or the branch you intentionally want to test).
☐ git status is clean before pulling or switching branches.
☐ dotnet --version shows .NET 8 SDK.
☐ LocalDB is installed and MSSQLLocalDB can start.
☐ Jwt:Key exists in .NET user-secrets on this PC.
☐ dotnet build reports Build succeeded.
☐ API starts without a fatal startup exception.
☐ Swagger opens on port 7239 or 5197.
☐ Development admin account is seeded (if AdminSeed config is present).
☐ frontend/js/config.js points to the actual backend API URL.
☐ Frontend opens through a static server and can call the API.

15. Command Cheat Sheet

```
# Clone develop
git clone -b develop https://github.com/CodeEver-CEO/Event-Parking-Reservation-System-Team.git
cd Event-Parking-Reservation-System-Team

# Configure JWT secret
cd .\backend\EventParkingReservationSystem.API
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Max 256 }))
dotnet user-secrets set "Jwt:Key" "PASTE_KEY_HERE"
cd ..\..

# Build
dotnet restore .\backend\EventParkingReservationSystem.sln
dotnet build .\backend\EventParkingReservationSystem.sln

# Run backend
dotnet run --project .\backend\EventParkingReservationSystem.API\EventParkingReservationSystem.API.csproj --launch-profile https

# Swagger
https://localhost:7239/swagger
http://localhost:5197/swagger

# Update later
git switch develop
git fetch origin
git pull --ff-only origin develop
```

Source Files Reviewed for This Guide

This guide was prepared from the uploaded develop-branch project snapshot, especially:

- README.md
- backend/EventParkingReservationSystem.API/EventParkingReservationSystem.API.csproj
- backend/EventParkingReservationSystem.API/appsettings.json
- backend/EventParkingReservationSystem.API/appsettings.Development.json
- backend/EventParkingReservationSystem.API/Properties/launchSettings.json
- backend/EventParkingReservationSystem.API/Program.cs
- backend/EventParkingReservationSystem.API/Data/ApplicationDbContext.cs
- backend/EventParkingReservationSystem.API/Data/DatabaseSeeder.cs
- frontend/js/config.js
- backend/EventParkingReservationSystem.API/Migrations/

Note: The repository can change after this snapshot. When handing the project to another developer, always pull the current target branch and compare its README, launchSettings.json, appsettings files and frontend API base URL.